



ProxySQL in Action: Architecting Scalable and Performant MySQL Environments

René Cannaò
Founder/CEO at ProxySQL LLC

MySQL BR Conf 2025 - 4th October 2025



<https://www.proxysql.com>

About The Speaker



René Cannaò

Founder & CEO of ProxySQL

Over 25 years of experience in database architecture & performance optimization

Former MySQL consultant and performance expert

Creator of ProxySQL to solve real-world MySQL scalability challenges

Passionate about high-performance databases, open-source development, and community collaboration



<https://www.proxysql.com>

MySQL BR Conf 2025

Agenda

The 30-Year Legacy of MySQL & Its Modern Scaling Challenges

Why Traditional Scaling Approaches Fall Short

Introducing ProxySQL: The Database-Aware Proxy

Deep Dive: Core Features in Action

Key Takeaways & Q&A.



<https://www.proxysql.com>

MySQL BR Conf 2025

MySQL at 30: From Disruptor to Enterprise Standard

Launched in 1995, it became the heart of the LAMP stack.

Evolved into a mission-critical RDBMS with the introduction of InnoDB.

The Oracle era brought a focus on enterprise-grade performance, security, and HA.

Key takeaway: Its success led to workloads at a scale never originally imagined.





The Scaling Problem Today

Explosive growth in both user and data volume.

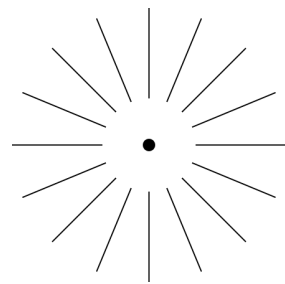
High concurrency from modern applications creates massive connection pressure.

Query latency directly impacts User Experience (UX) and business KPIs.

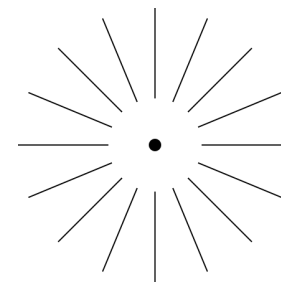
Complex topologies are now the norm, with high availability as a baseline requirement.



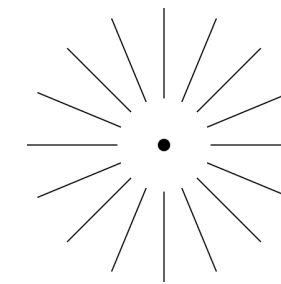
Architectural Hurdle #1: The Connection Bottleneck



MySQL's architecture uses a
“one-thread-per-connection”
model.



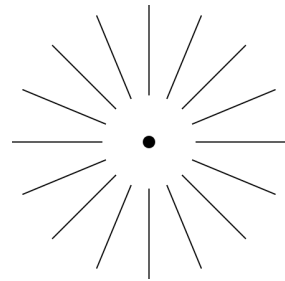
Every client connection, even
idle ones, consumes a
dedicated MySQL thread,
memory, and CPU resources.



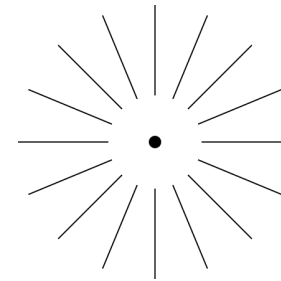
Spikes in traffic (from app
restarts, scaling events) can
lead to CPU/memory
exhaustion and database
unresponsiveness.



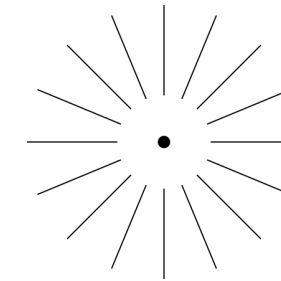
Architectural Hurdle #2: Operational Fragility



Asynchronous replication can lead to "stale reads" if replicas lag behind the primary.



This eventual consistency can break the user experience with read-after-write anomalies.



Database failover is often a manual, high-risk process that causes prolonged downtime.



Architectural Hurdle #3:

Application-Level Complexity

Logic for read/write splitting must be built into the application code.

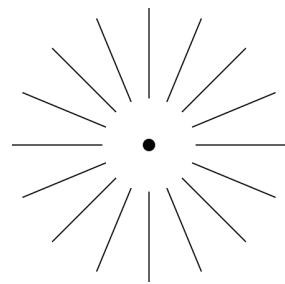
This creates a tight coupling between the application and the database topology.

This complex logic must be replicated and maintained across all microservices.

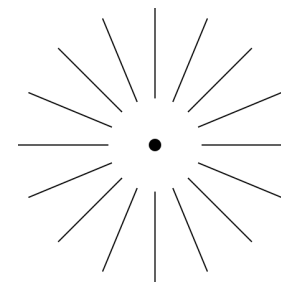
Infrastructure changes (like adding a replica) often require application code changes and redeployments.



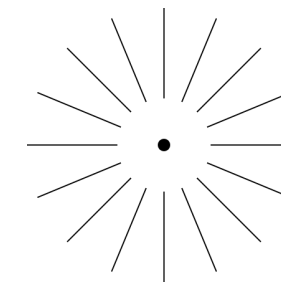
Why Traditional Approaches Fall Short



Native MySQL:
Lacks built-in load balancing
and automated failover.



L3/L4 Proxies (HAProxy, LVS):
Are not SQL-aware. They
cannot inspect queries to
perform intelligent routing,
read/write splitting, or
caching.



Application Logic:
As we have seen, this is
complex, brittle, and couples
the app to the infrastructure.



Enter ProxySQL: A Purpose-Built Solution



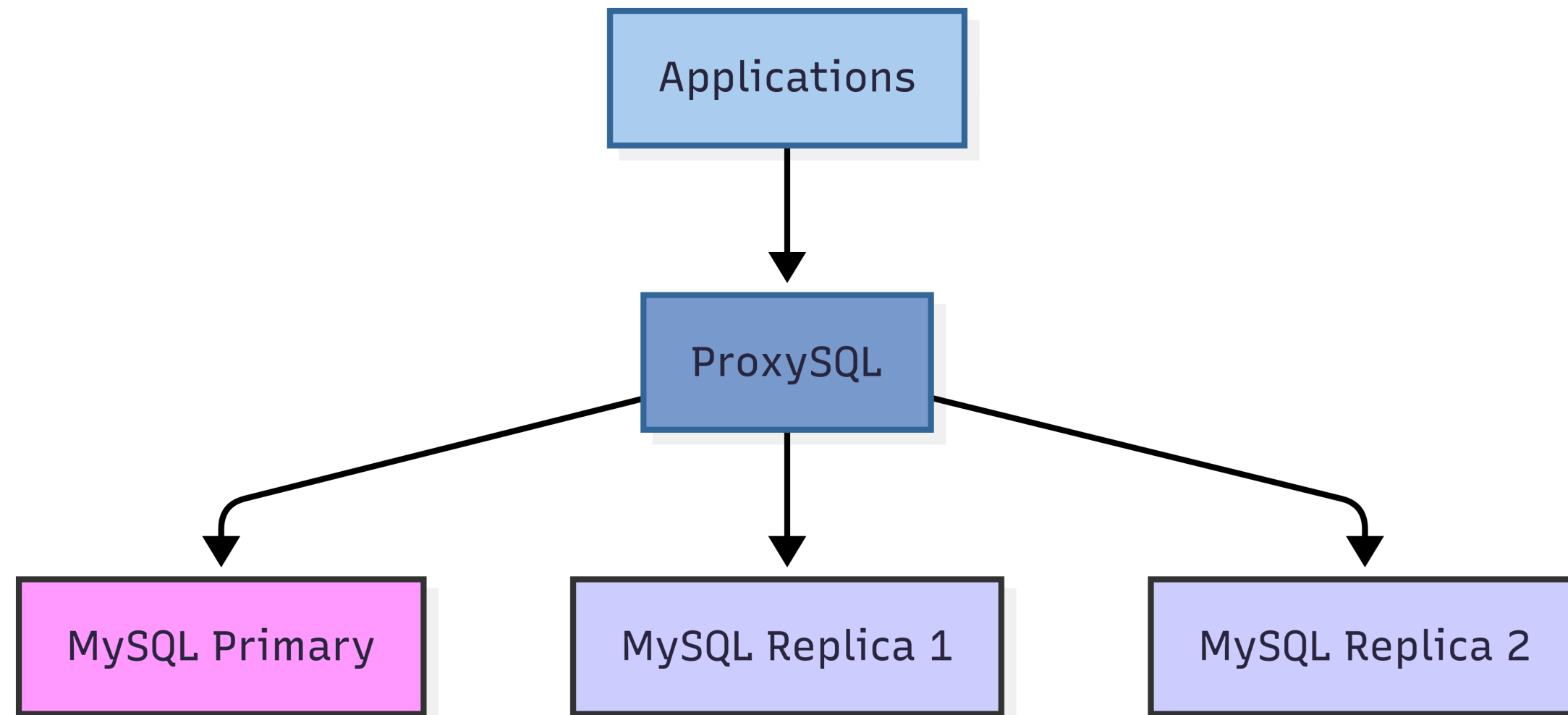
A high-performance,
open-source, SQL-aware
proxy for MySQL.

Sits transparently between
your applications and the
MySQL backend cluster.

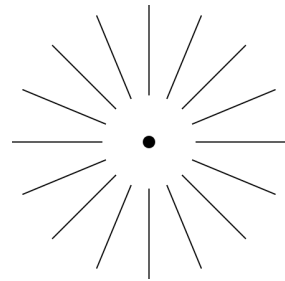
Its core purpose is to solve
MySQL's scaling limitations
by intelligently managing the
query lifecycle.



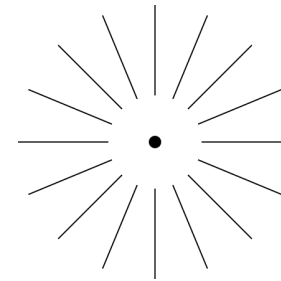
ProxySQL: An Intelligent Layer for MySQL



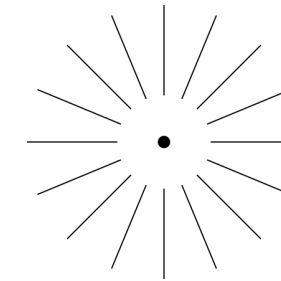
The Core Principle: Decoupling



ProxySQL acts as a layer of abstraction between developers and operators.



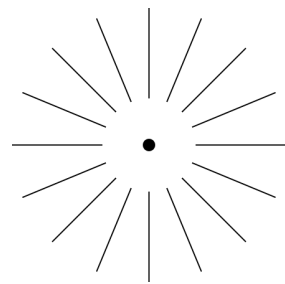
Developers focus on business logic and connect to a single ProxySQL endpoint.



Operators can manage the backend topology (add replicas, perform failovers, migrate servers) transparently, without impacting the application.

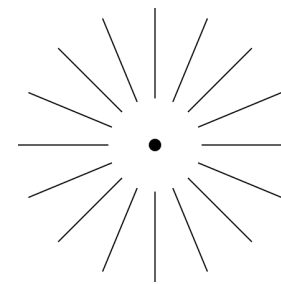


Feature: Connection Multiplexing



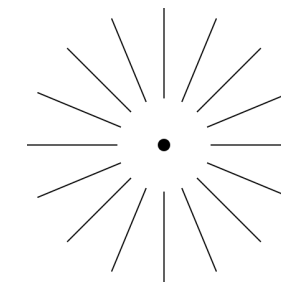
Problem:

Connection storms and thread exhaustion.



Solution:

ProxySQL consolidates thousands of frontend connections into a small, efficient pool of backend connections. It reuses idle connections, drastically reducing overhead on MySQL threads and memory.

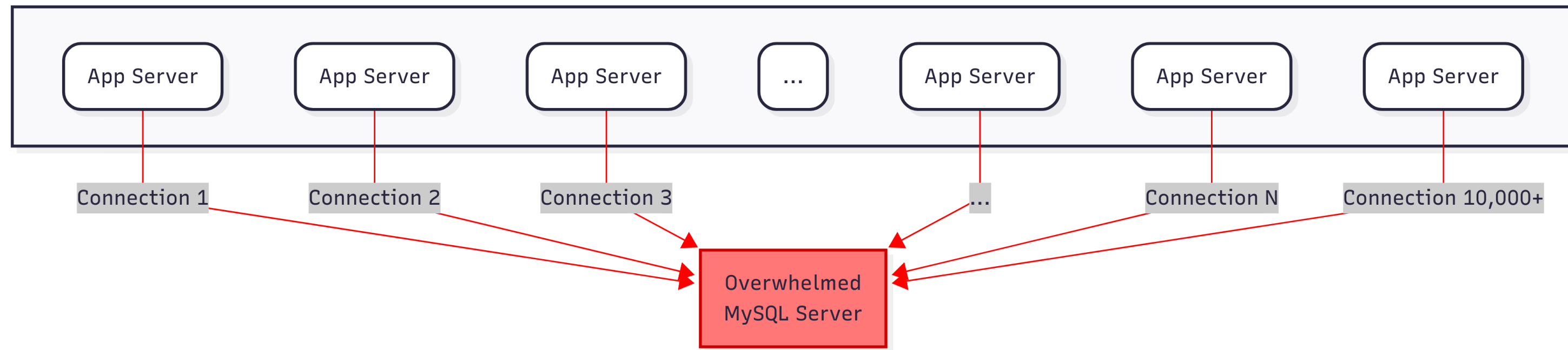


Result:

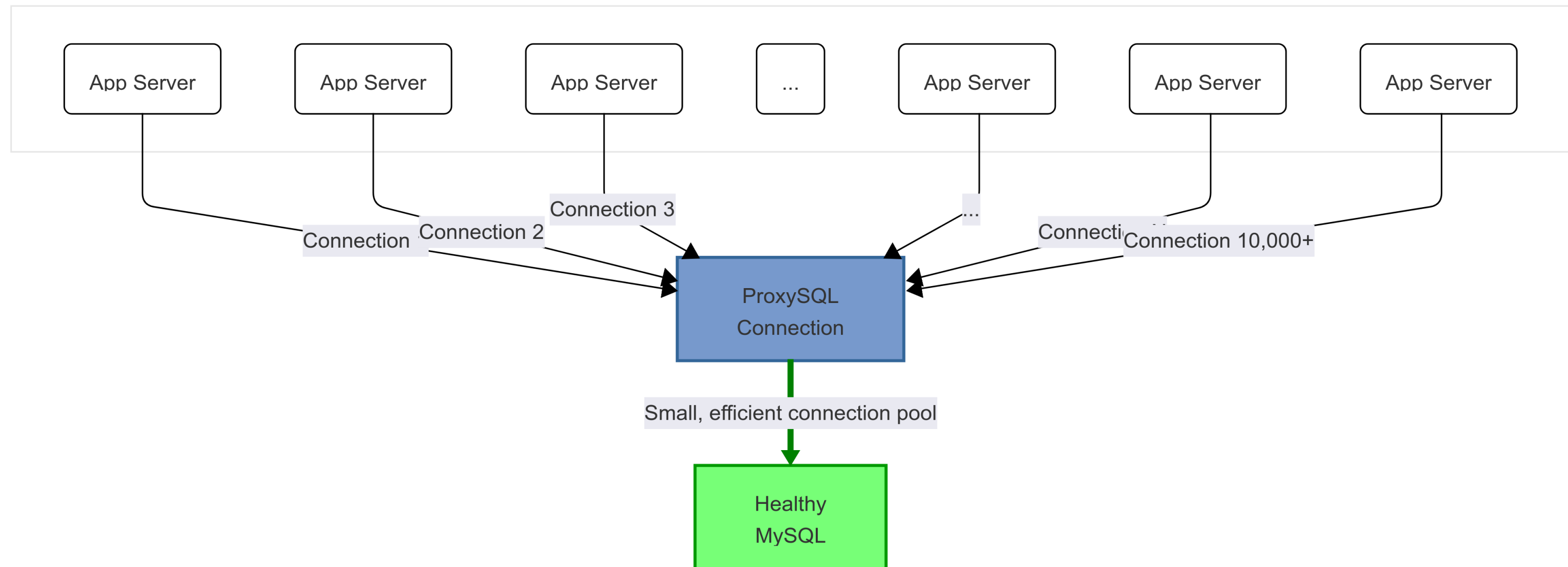
Reduces backend connections by up to 100x and handles traffic bursts without overloading the database.



Multiplexing in Action (Before ProxySQL)

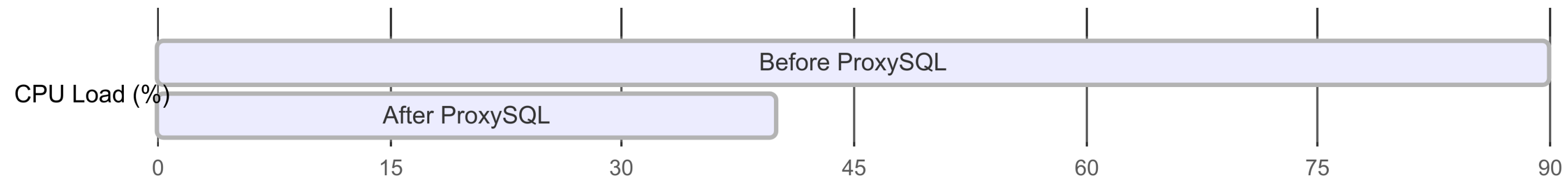
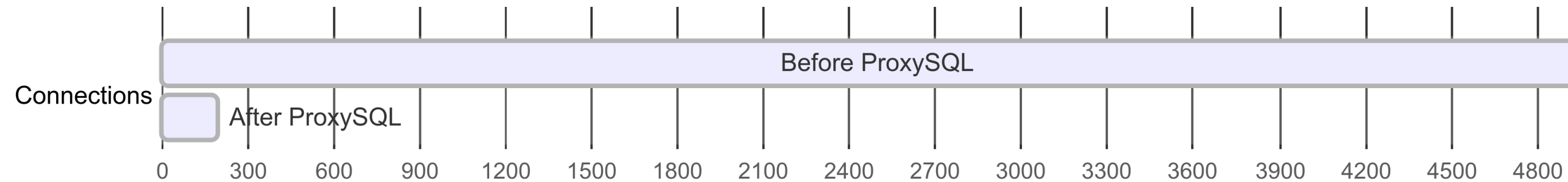


Multiplexing in Action (After ProxySQL)

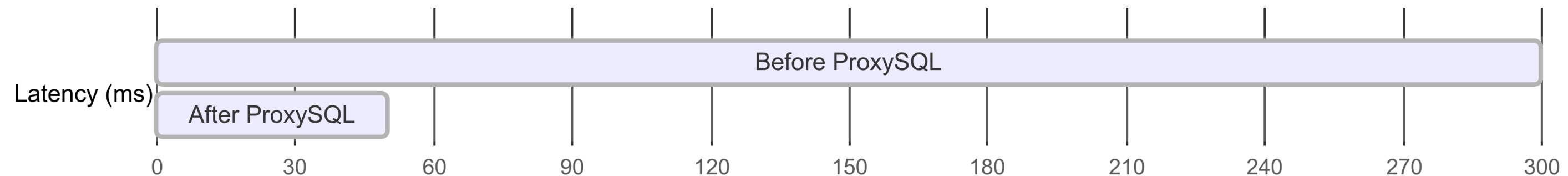


Multiplexing in Action (Result)

Impact: Database Connections



Impact: P99 Query Latency



Feature: Query Routing

Problem:

Query routing chaos, with complex logic spread across applications.

Solution:

ProxySQL implements a powerful, ordered rules engine to control traffic flow.

Details:

It uses two key concepts:

- 1) **Hostgroups**: Logically group servers by role (e.g., writers, readers).
- 2) **Query Rules**: The "if-then" logic that sends queries to the correct hostgroup.



Query Routing in Action: Read/Write Split

This is the most common use case.

With a few simple rules, you can offload all/most your read traffic.

Example ``mysql_query_rules``:

Rule 1: ``match_pattern: ^SELECT .* FOR UPDATE`, `destination_hostgroup: 10 (Writers)``

Rule 2: ``match_pattern: ^SELECT`, `destination_hostgroup: 20 (Readers)``

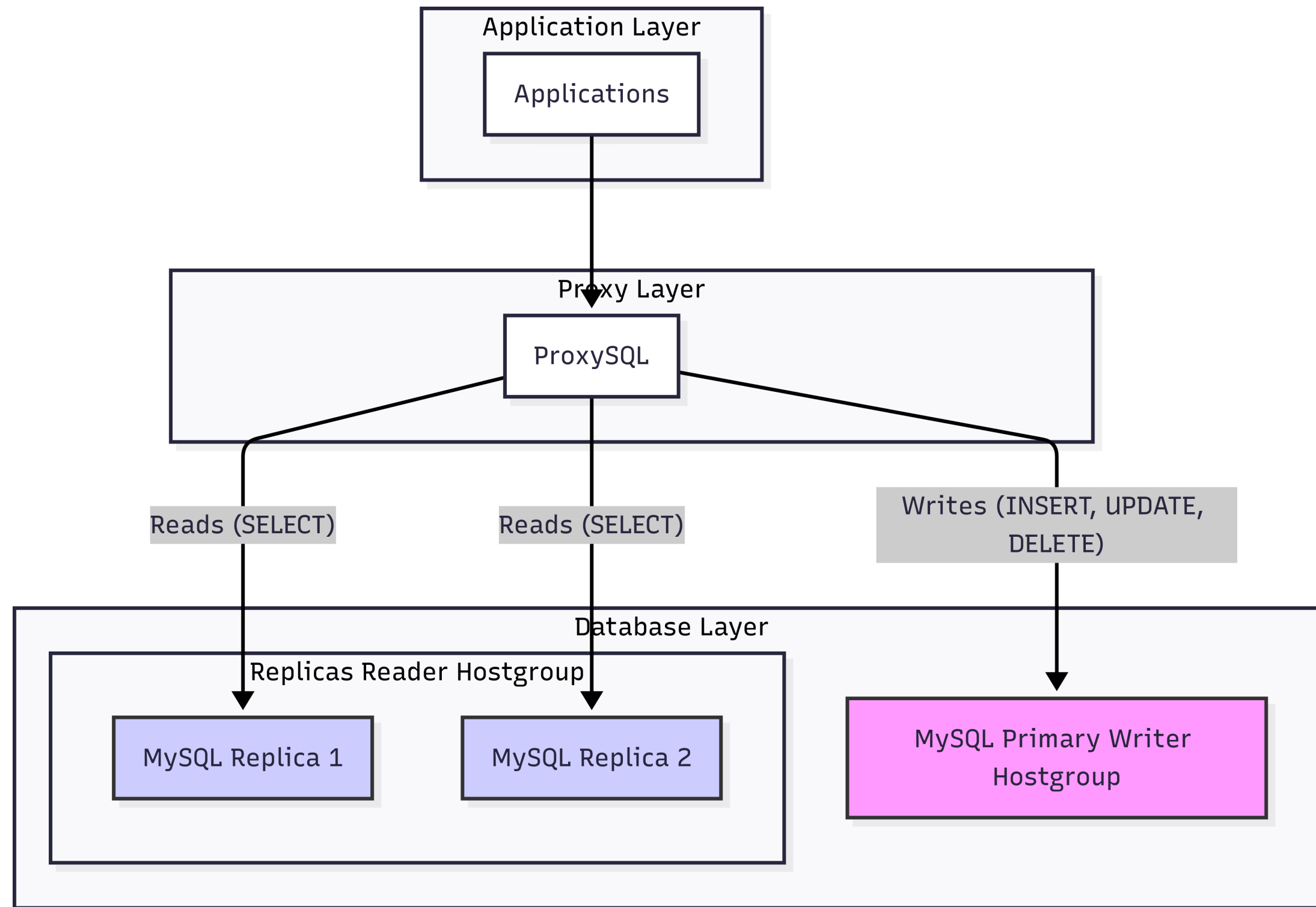
Default: All other queries (INSERT, UPDATE) go to the writer hostgroup (10).

Benefit: Achieved without a single application code change.

DISCLAIMER: for illustration purposes only, do not use it in production.



Query Routing in Action: Read/Write Split





Feature: High Availability

Problem: A manual failover process leads to prolonged and stressful downtime.

Solution: ProxySQL provides automated failover and failback.

It integrates seamlessly with native MySQL HA solutions like Group Replication, Galera, and more.

it constantly monitors the health of all backend MySQL nodes using built-in or custom health checks.

HA in Action: The Automated Failover Process

Failure:

The primary node crashes.

Election:

The cluster (e.g., Group Replication) elects a new primary in seconds.
For standard replication, the new primary sets
``read_only = 0``.

Detection:

ProxySQL's monitoring sees the old primary is offline and the new primary is now writable.

Rerouting:

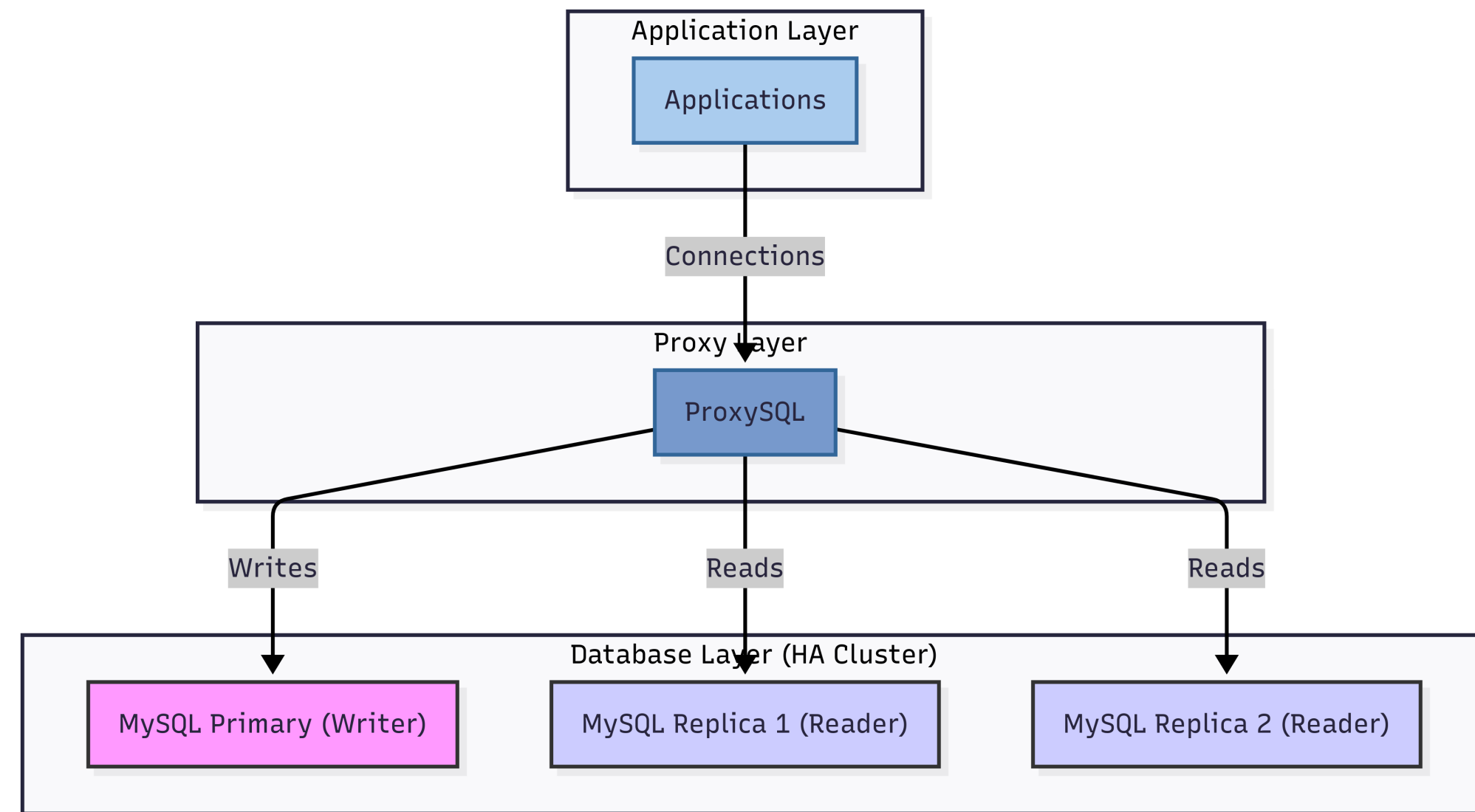
ProxySQL automatically moves the new primary into the writer hostgroup.

Seamless Transition:

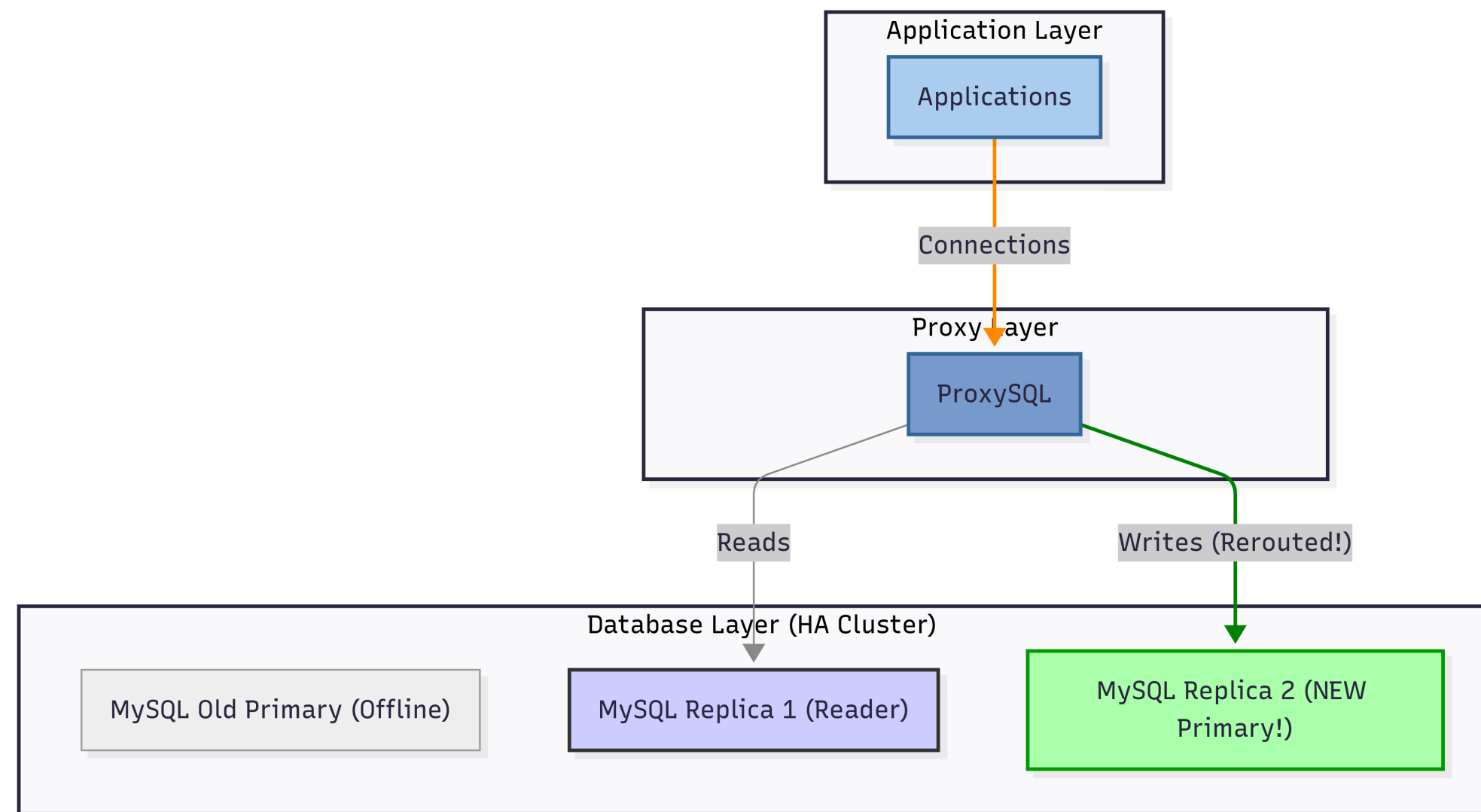
Application writes are transparently sent to the new primary with no manual intervention required.



HA in Action: The Automated Failover Process



HA in Action: The Automated Failover Process



HA Bonus Feature: Split-Brain Prevention



ProxySQL has a built-in mechanism to prevent dangerous "split-brain" scenarios where multiple servers could accept writes.



It ensures that only one hostgroup can be the "writer" at any given time, protecting data integrity during chaotic network events.



Feature: Handling Replication Lag

Problem:

Reading from a lagging replica can return stale data, breaking consistency assumptions.

Solution:

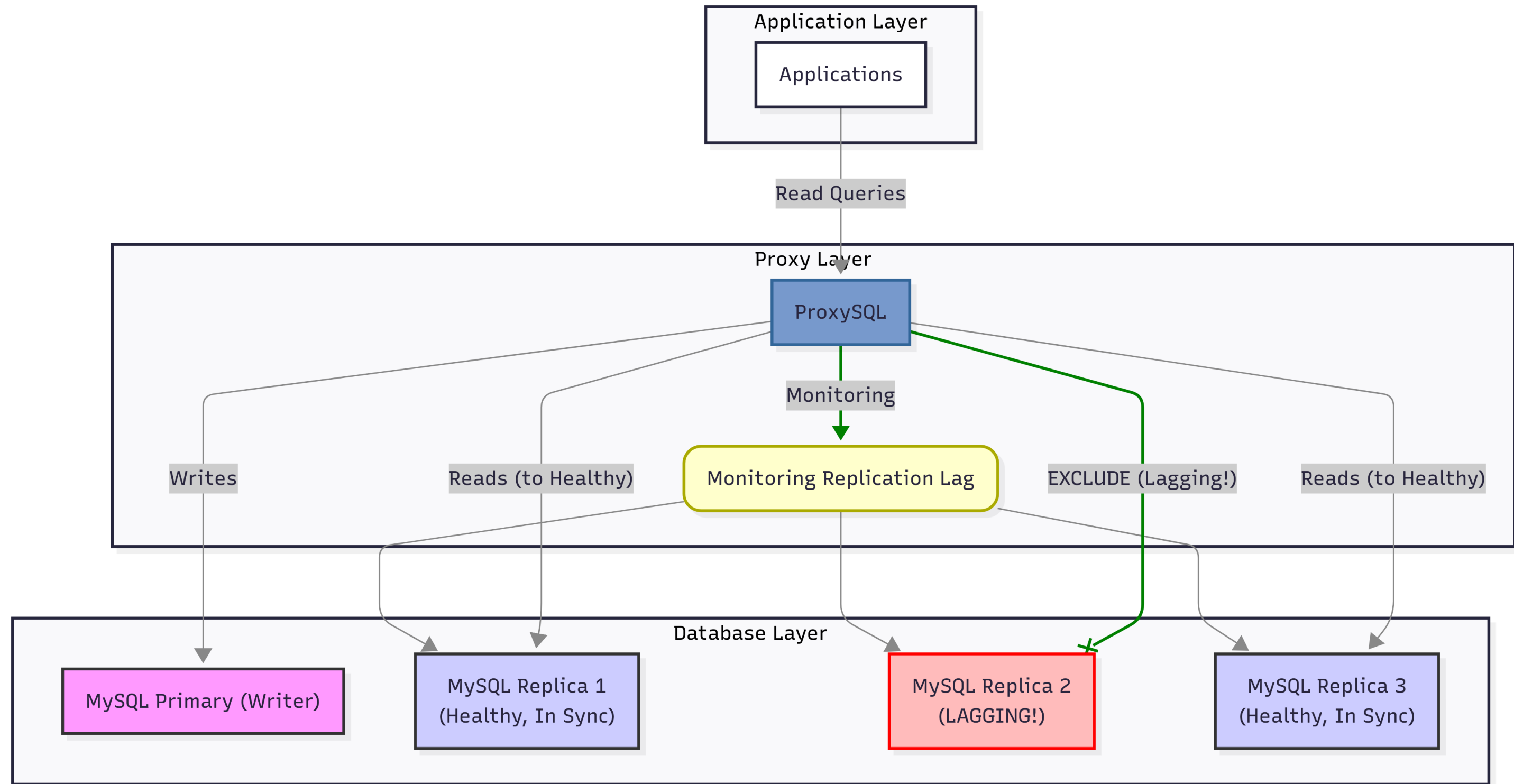
ProxySQL performs real-time monitoring of replication lag on every replica node.

It can be configured to only route read queries to healthy, up-to-date replicas that are within a specified lag threshold.

This effectively avoids stale reads during failovers or lag spikes.



Feature: Handling Replication Lag



Feature: Query Rewriting

The Power: Fix bad queries without touching application code.

A powerful tool for DBAs and SREs for incident response and tactical optimization.

Use Cases:

1. Rewrite slow or unsafe queries in-flight (e.g., add ``LIMIT``, change ``SELECT *``).
2. Add index hints to force a better execution plan.
3. Useful for patching legacy apps, managing migrations, or applying hotfixes.



Query Rewriting in Action

The Problem:

1

A legacy application is running a table scan.

```
`SELECT * FROM users WHERE email = 'test@example.com';`
```

The Solution in ProxySQL:

2

rule in `mysql\_query\_rules`:

```
`match_pattern: "SELECT \* FROM users WHERE email"``
```

```
`replace_pattern: "SELECT id, name, email FROM users USE  
INDEX(idx_email) WHERE email"``
```

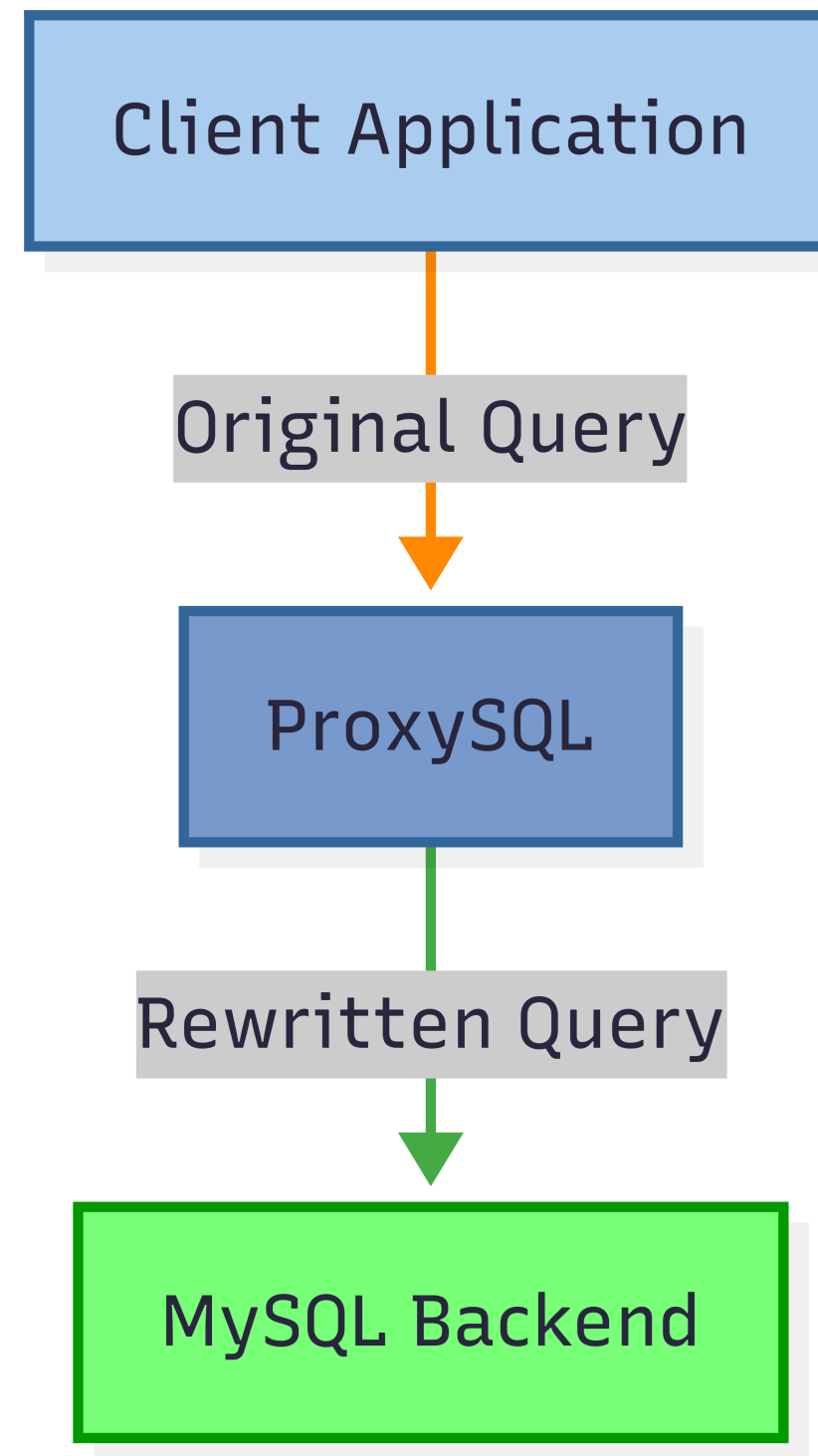
3

Result:

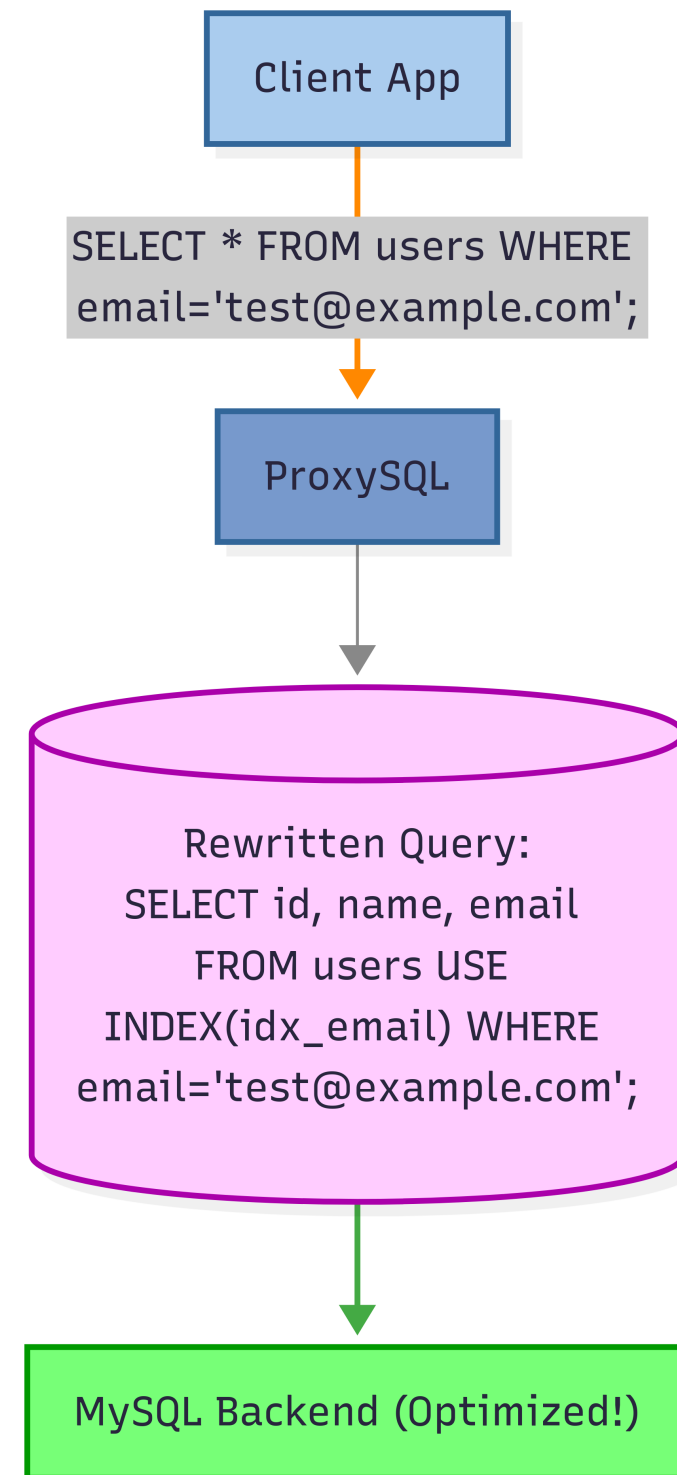
Performance is restored instantly, no deployment needed.



Query Rewriting in Action



Query Rewriting in Action



Feature:

Query Caching

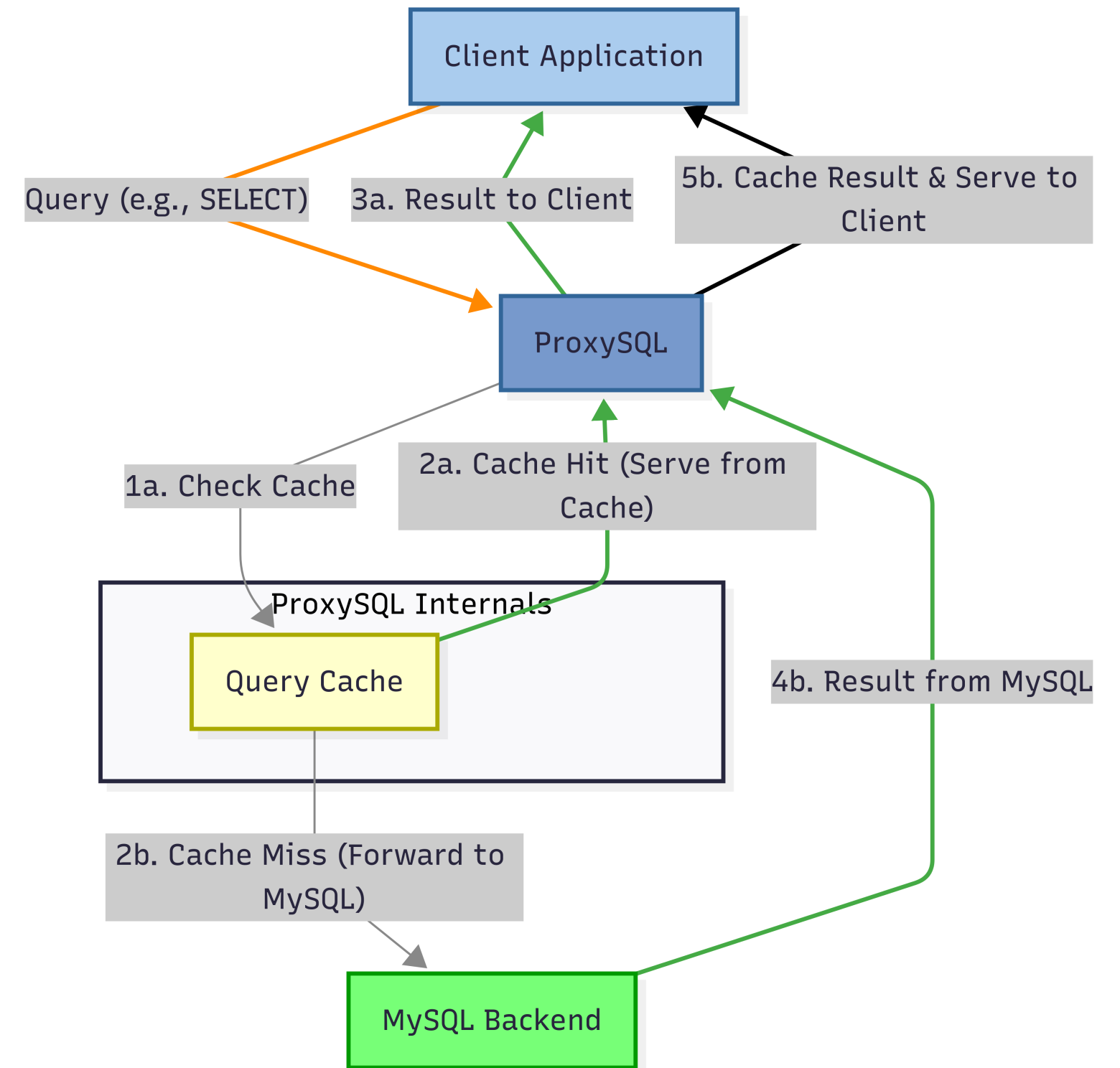
The Goal:

Offload repetitive queries and serve results instantly.

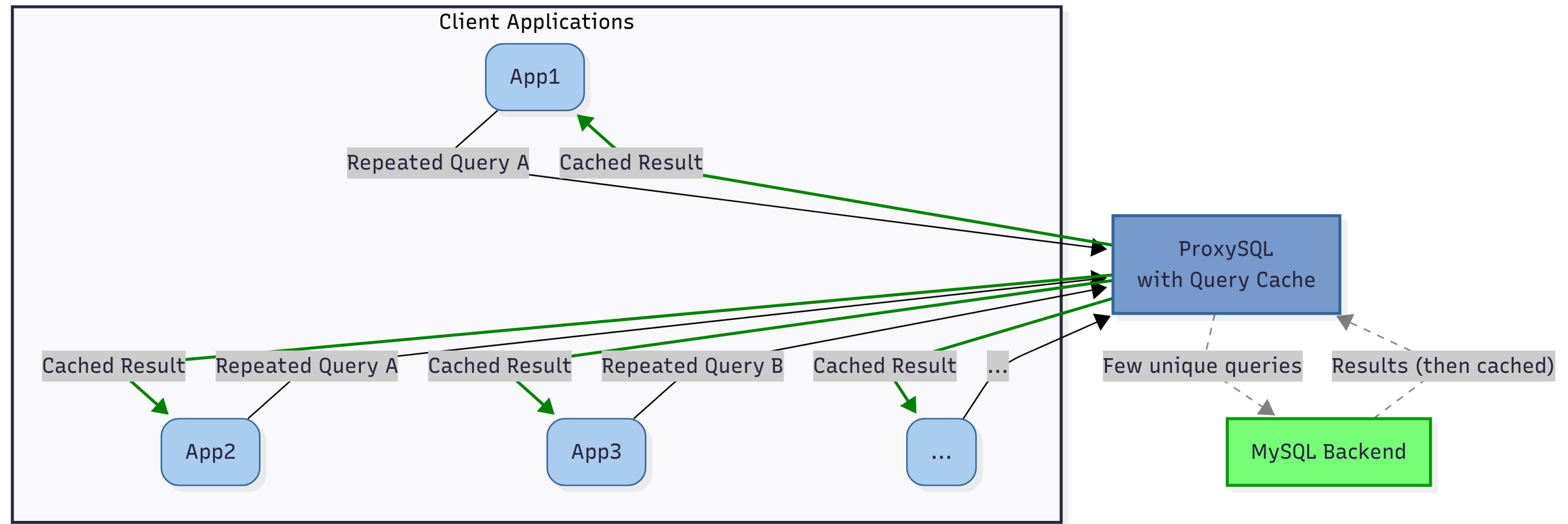
- It provides a high-performance, in-memory, rule-based SQL result set cache.
- It replaces MySQL's deprecated query cache with a much more efficient and controllable solution.
- Delivers blazing-fast, sub-millisecond response times for repeated queries. Reduces backend CPU and disk I/O load.



Feature: Query Caching



Feature: Query Caching





Modern **database scaling platform** and **SQL query caching** system.

Boost performance, optimize infrastructure costs, and improve engineering velocity with proactive enhancements for PostgreSQL & MySQL

Reduce hosting costs **by up to 70%**

In memory query read cache - sub millisecond

Auto synced to source db changes **in milliseconds**

No **TTLs**, no **cache invalidations**

Unchanged cache latency even when **data changes**



Available through

aws marketplace

Feature:

Observability & Real-Time Stats

Problem to solve:

Without a proxy, you have no visibility into query patterns, timings, or routing decisions.

Solution:

ProxySQL provides deep observability into your database traffic.



It has built-in performance counters and query analytics.



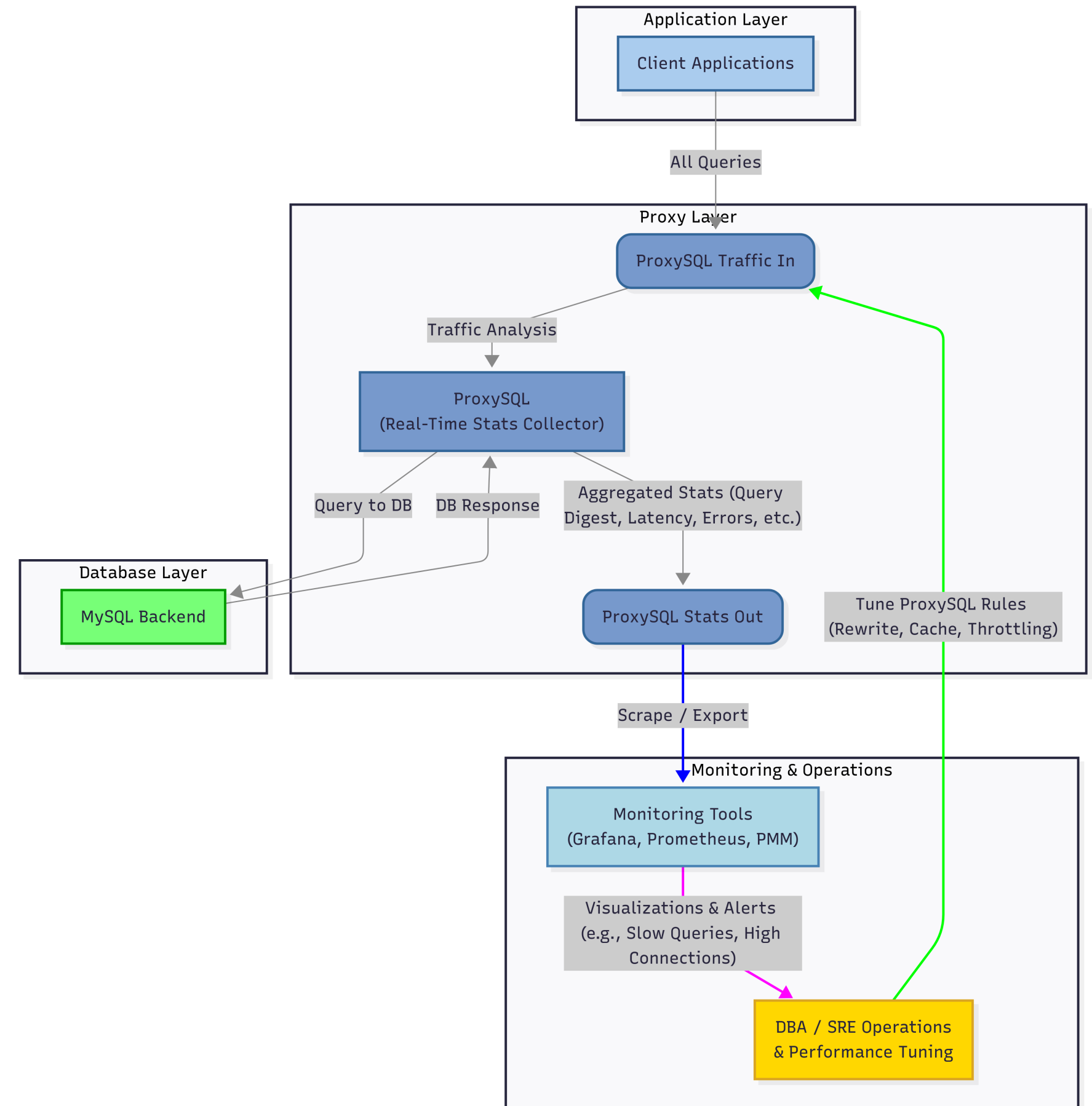
You get a real-time query digest with histograms to identify slow or frequent queries.



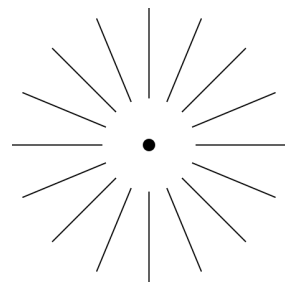
It easily integrates with monitoring tools like Grafana, Prometheus, and PMM.



Feature: Observability & Real-Time Stats

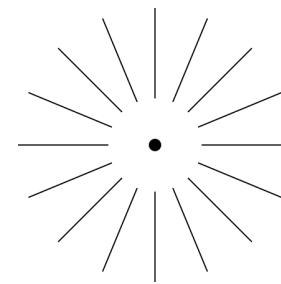


Feature: Traffic Shaping & Control



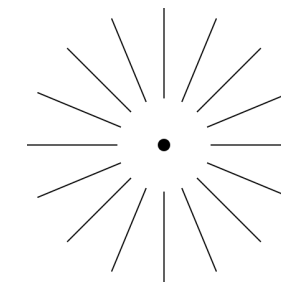
Query Throttling:

Limit the execution rate of specific query patterns to prevent "noisy neighbor" problems.



Advanced Firewall:

Apply traffic policies by user, source IP address, query pattern, others



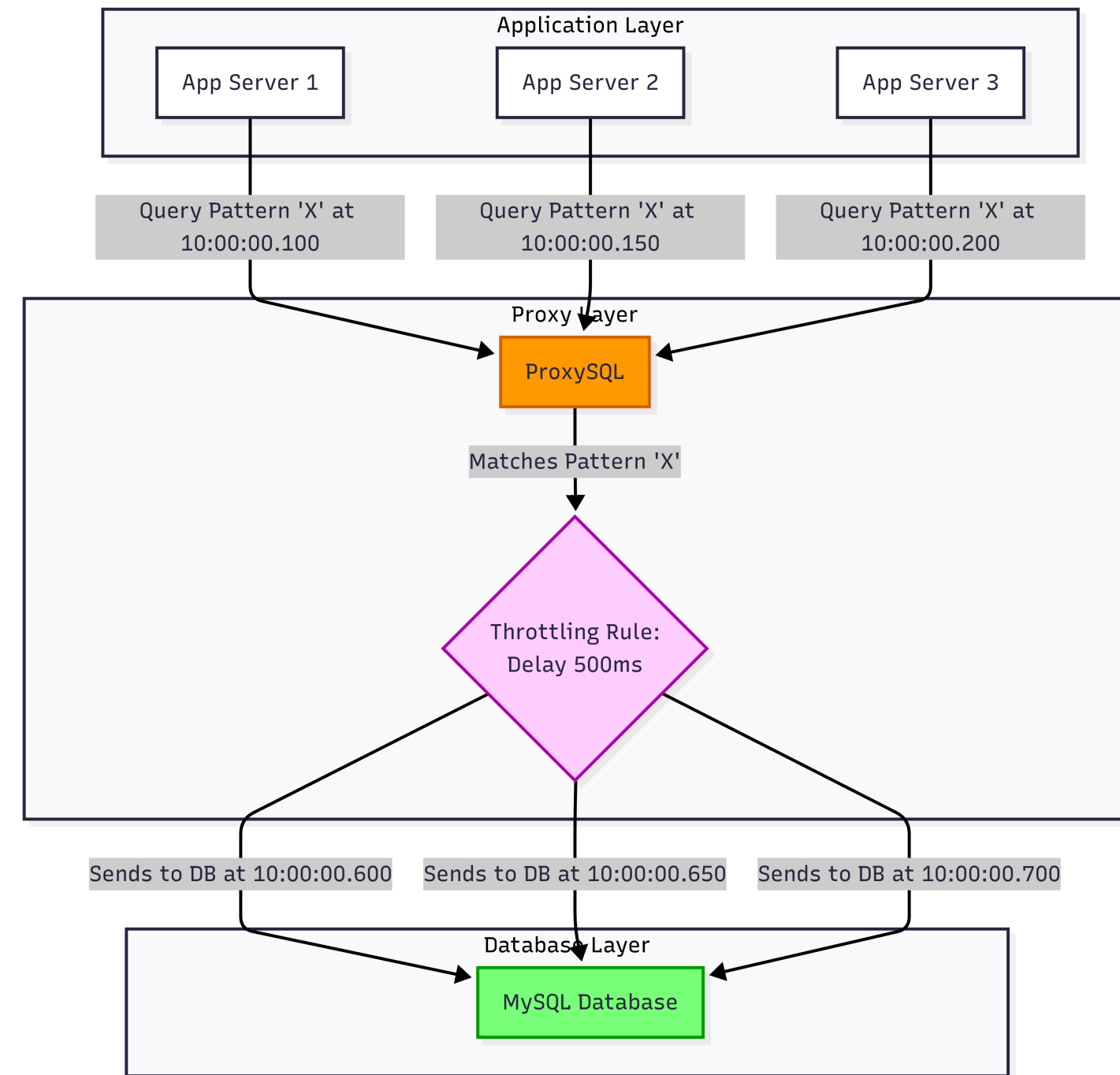
Security:

Provides advanced authentication options and data masking capabilities to hide sensitive data.



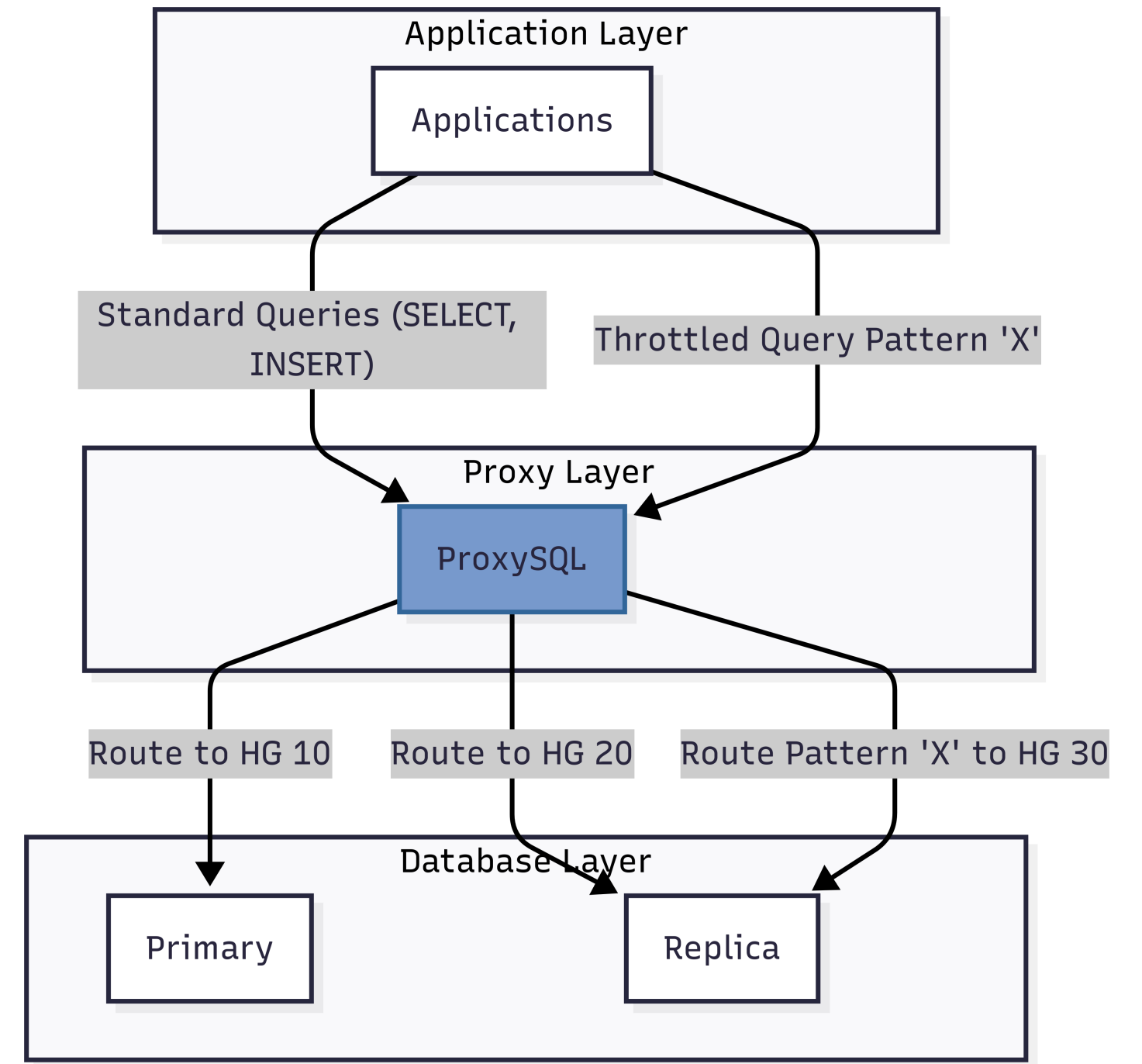
Feature: Traffic Shaping & Control

Query Throttling: delay



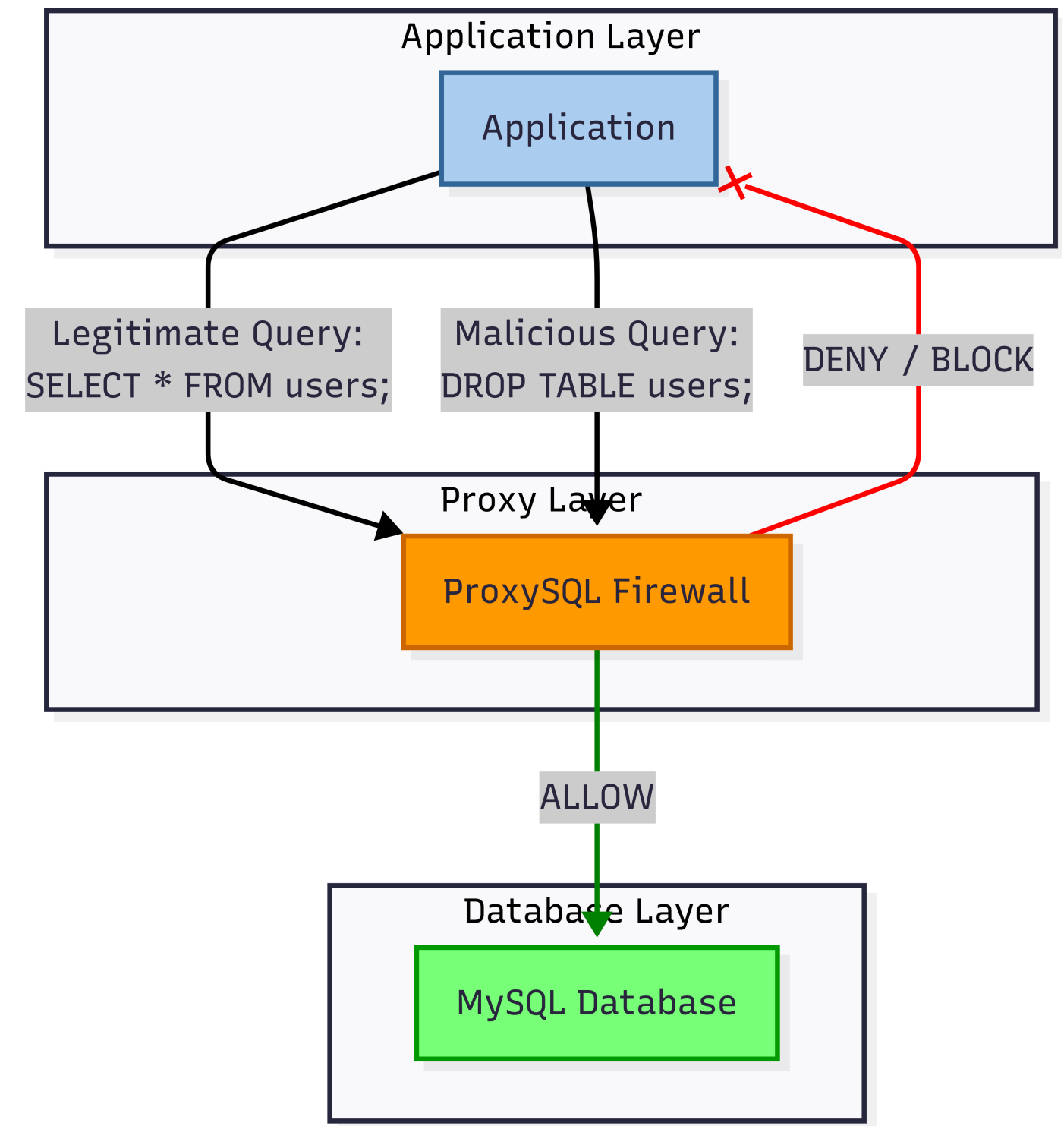
Feature: Traffic Shaping & Control

Query Throttling: connections



Feature: Traffic Shaping & Control

Firewall



Real-World Use Cases

01

Classic HA Read/Write splitting with standard replication.

02

Managing complex InnoDB Cluster, Galera, and multi-primary setups.

03

Multi-region and hybrid cloud database architectures.

04

Demanding workloads: SaaS, E-commerce, FinTech, and Gaming.

05

Managing huge infrastructures with thousands of database clusters.



Why Choose ProxySQL?

Trusted by companies from startups to Fortune 500s

01

Scalability: Handle hundreds of thousands of connections, and thousands of backend.

02

Performance: Reduce query latency and backend load.

03

Reliability: Built-in automated failover and high availability.

04

Observability: Real-time query analytics and monitoring.

05

Easy Adoption: No application changes are needed to get started.

06

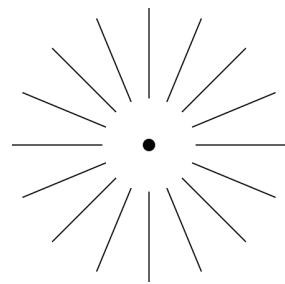
Security: Advanced authentication , traffic filtering and data masking



<https://www.proxysql.com>

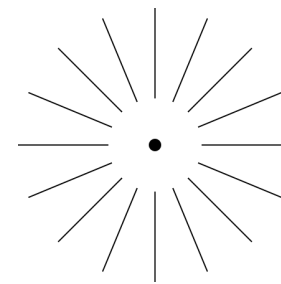
MySQL BR Conf 2025

Key Takeaways



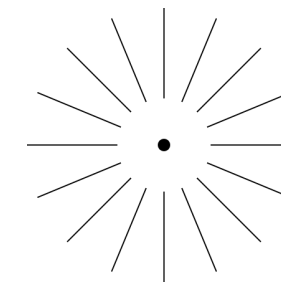
Decouple:

Abstract your application from your database topology.



Centralize:

Manage scaling challenges like connection handling, routing, and HA in a single, intelligent layer.



Optimize:

Gain powerful operational control to rewrite, cache, and throttle queries without touching application code.



Questions?

Add more text

- Happy to go deeper on any topic.
- Open to questions, ideas, and suggestions.



Thank You

Join the Community

GitHub:

<https://github.com/sysown/proxysql>

Website:

<https://proxysql.com/>

Documentation:

<https://proxysql.com/documentation/>

email:

rene@proxysql.com



LinkedIn



<https://www.proxysql.com>

MySQL BR Conf 2025